

LAB ASSIGNMENT – 2.5

NAME : SHREE PRIYA

2303A51465

BATCH - 29

Task 1: Refactoring Odd/Even Logic (List Version)

❖ Scenario:

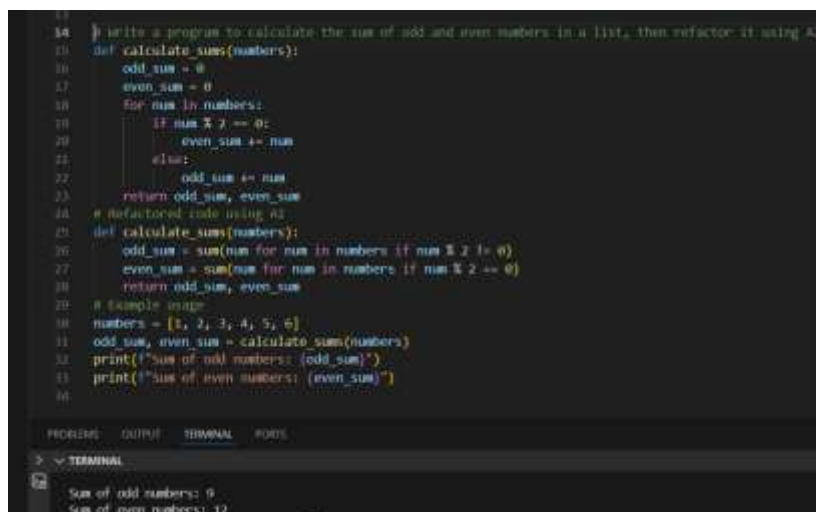
You are improving legacy code.

❖ Task:

Write a program to calculate the sum of odd and even numbers in a list, then refactor it using AI.

❖ Expected Output:

❖ Original and improved code



```
14 # write a program to calculate the sum of odd and even numbers in a list, then refactor it using AI
15 def calculate_sums(numbers):
16     odd_sum = 0
17     even_sum = 0
18     for num in numbers:
19         if num % 2 == 0:
20             even_sum += num
21         else:
22             odd_sum += num
23     return odd_sum, even_sum
24 # refactored code using AI
25 def calculate_sums(numbers):
26     odd_sum = sum(num for num in numbers if num % 2 != 0)
27     even_sum = sum(num for num in numbers if num % 2 == 0)
28     return odd_sum, even_sum
29 # sample usage
30 numbers = [1, 2, 3, 4, 5, 6]
31 odd_sum, even_sum = calculate_sums(numbers)
32 print(f"Sum of odd numbers: {odd_sum}")
33 print(f"Sum of even numbers: {even_sum}")
34
```

PROBLEMS OUTPUT TERMINAL PORTS

TERMINAL

Sum of odd numbers: 9
Sum of even numbers: 12

Task 2: Area Calculation Explanation

❖ Scenario:

You are onboarding a junior developer.

❖ Task:

Ask Gemini to explain a function that calculates the area of different shapes.

❖ Expected Output:

➤ Code

➤ Explanation

```
29
30
31 def calculate_area(shape, dimensions):
32     if shape == "rectangle":
33         length, width = dimensions
34         return length * width
35     elif shape == "circle":
36         radius = dimensions
37         return 3.14159 * radius * radius
38     elif shape == "triangle":
39         base, height = dimensions
40         return 0.5 * base * height
41     else:
42         raise ValueError("Unknown shape")
43
44 # Example usage
45 print(calculate_area("rectangle", (5, 10)))
46 print(calculate_area("circle", (7,)))
47 print(calculate_area("triangle", (4, 8)))
48
```

PROBLEMS OUTPUT TERMINAL PORTS

> TERMINAL

```
50
153.93791
16.0
```

Task 3: Prompt Sensitivity Experiment

❖ Scenario:

You are testing how AI responds to different prompts.

❖ Task:

Use Cursor AI with different prompts for the same problem and observe code changes.

❖ Expected Output:

➤ Prompt list

➤ Code variations

```
46
47 def calculate_factorial(number):
48     factorial_result = 1
49     for i in range(1, number + 1):
50         factorial_result *= i
51     return factorial_result
52
53 # Example usage
54 print(calculate_factorial(5))
55
```

PROBLEMS OUTPUT TERMINAL PORTS

> TERMINAL

```
120
```

Task 4: Tool Comparison Reflection

❖ Scenario:

You must recommend an AI coding tool.

❖ Task:

Based on your work in this topic, compare Gemini, Copilot, and Cursor AI for usability and code quality.

❖ Expected Output:

Short written reflection

```
# I used Gemini, then Github Copilot, and lastly Cursor AI. Although Gemini is helpful in explanation purposes,  
# it sometimes requires some sort of correction in code. But Github Copilot is indeed super helpful,  
# as it can be used within an editor, thus speeding up coding. Nevertheless, Copilot may sometimes fail to grasp  
# complex instructions. On the contrary, Cursor AI is way better in terms of usability, as it grasps  
# instructions perfectly, thus ensuring clean code. I will definitely recommend Cursor AI to students due to usability purposes.  
|
```